

ITF23019: Parallel and Distributed Programming

Report for Lab 6: Thread Synchronization 1

Name: Emil Berglund

GitHub: EmilB04

Exercise 1:

- Subtask 1: Run the code several times and include the output
 - *Attached 3 screenshots:*

```

● Parallel-distributed-programming/lab6 on 🐚 main > /usr/bin/
re/21.0.9-macosx-aarch64/bin/java -XX:+ShowCodeDetailsInExce
buted-programming/lab6/lab6_code/target/classes emilbe.Main
Thread-0: 2
Thread-1: 10
Thread-1: 20
Thread-0: 4
Thread-1: 30
Thread-0: 6
Thread-1: 40
Thread-0: 8
Thread-1: 50
Thread-0: 10
Thread-1: 60
Thread-0: 12
Thread-1: 70
Thread-0: 14
Thread-1: 80
Thread-0: 16
Thread-1: 90
Thread-0: 18
Thread-1: 100
Thread-0: 20
🐚
○ Parallel-distributed-programming/lab6 on 🐚 main > |

```

```

● Parallel-distributed-programming/lab6 on 🐚 main >
rogramming/lab6 ; /usr/bin/env /Users/emilb/.vscode
-aarch64/bin/java -XX:+ShowCodeDetailsInExceptionMe
ibuted-programming/lab6/lab6_code/target/classes em
Thread-1: 10
Thread-0: 2
Thread-1: 20
Thread-0: 4
Thread-1: 30
Thread-0: 6
Thread-0: 8
Thread-1: 40
Thread-1: 50
Thread-0: 10
Thread-1: 60
Thread-0: 12
Thread-0: 14
Thread-1: 70
Thread-1: 80
Thread-0: 16
Thread-0: 18
Thread-1: 90
Thread-0: 20
Thread-1: 100
🐚
○ Parallel-distributed-programming/lab6 on 🐚 main > |

```

```

● Parallel-distributed-programming/lab6 on 🐚 main > /
.0-darwin-arm64/jre/21.0.9-macosx-aarch64/bin/java -
ments/GitHub/V2026/Parallel-distributed-programming/
d
Thread-0: 2
Thread-1: 10
Thread-1: 20
Thread-0: 4
Thread-0: 6
Thread-1: 30
Thread-0: 8
Thread-1: 40
Thread-0: 10
Thread-1: 50
Thread-0: 12
Thread-1: 60
Thread-0: 14
Thread-1: 70
Thread-0: 16
Thread-1: 80
Thread-0: 18
Thread-1: 90
Thread-0: 20
Thread-1: 100
🐚
○ Parallel-distributed-programming/lab6 on 🐚 main > |

```

- Subtask 2: Do you get the same output from all runs?
 - No, I do not get the same output each time. The key difference is the execution order varies between runs. This demonstrates non-deterministic behaviour due to threads not being synchronized.
- Subtask 3: Add synchronized keyword to synchronize two threads.
 - After adding the keyword *synchronized*, I always got the same output for all runs. On the other hand, when the code is synchronized it is approximately twice as slow as unsynchronized.

```
● Parallel-distributed-programming/lab6 on 🍏 main >
allel-distributed-programming/lab6 ; /usr/bin/env
-1.52.0-darwin-arm64/jre/21.0.9-macosx-aarch64/bin
es -cp /Users/emilb/Documents/GitHub/V2026/Paralle
rget/classes emilbe.MonitorSynchronizeMethod
Thread-0: 2
Thread-0: 4
Thread-0: 6
Thread-0: 8
Thread-0: 10
Thread-0: 12
Thread-0: 14
Thread-0: 16
Thread-0: 18
Thread-0: 20
Thread-1: 10
Thread-1: 20
Thread-1: 30
Thread-1: 40
Thread-1: 50
Thread-1: 60
Thread-1: 70
Thread-1: 80
Thread-1: 90
Thread-1: 100
○ Parallel-distributed-programming/lab6 on 🍏 main >
```

Exercise 2: Refer to project lab6

- Subtask 1: Run package *producerConsumer* and include output
 - o Due to the length of the output, I will not include the whole console. Below are two screenshots, one showing the beginning, and one showing the end.

```
Parallel-distributed-programming/lab6 on J main > cd /Users/emilb/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin
+ShowCodeDetailsInExceptionMessages -cp /Users/emilb/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin
Support/Code/User/workspaceStorage/2335ad49a886886ea66d0at.java/jdt_ws/lab6_c8ffa991/bin producerConsumer.Main
Put: 1
Put: 2
Put: 3
Put: 4
Put: 5
Put: 6
Put: 7
Put: 8
Put: 9
Put: 10
Put: 11
Put: 12
Put: 13
Put: 14
Put: 15
Put: 16
Put: 17
Put: 18
Put: 19
Put: 20
Put: 21
Put: 22
Put: 23
Put: 24
Put: 25
Put: 26
Put: 27
Put: 28
Put: 29
Put: 30
```

```
Remove: 32:
Remove: 31:
Remove: 30:
Remove: 29:
Remove: 28:
Remove: 27:
Remove: 26:
Remove: 25:
Remove: 24:
Remove: 23:
Remove: 22:
Remove: 21:
Remove: 20:
Remove: 19:
Remove: 18:
Remove: 17:
Remove: 16:
Remove: 15:
Remove: 14:
Remove: 13:
Remove: 12:
Remove: 11:
Remove: 10:
Remove: 9:
Remove: 8:
Remove: 7:
Remove: 6:
Remove: 5:
Remove: 4:
Remove: 3:
Remove: 2:
Remove: 1:
Remove: 0:
Parallel-distributed-programming/lab6 on J main >
```

- Subtask 2: Which synchronization mechanism(s) and method(s) are used to synchronize the Producer thread and the Consumer thread in this package?
 - Consumer and Producer are utilizing a queue buffer class which declare methods such as *put* and *remove* which are both synchronized.
- Subtask 3: Remove the synchronization of Producer and Consumer threads and explain how a data race occurs

Exercise 3:

- Subtask 1: Explain why data race occurs
- Subtask 2: Where are the critical sections
- Subtask 3: Fix the data inconsistency problem using Monitor
- Subtask 4: Fix the data inconsistency using Lock

Exercise 4:

- Subtask 1: Run the two implementations – include outputs
- Subtask 2: Expected output of *numberCars* and *numberMotorcycles*.
- Subtask 3: Discuss the data race issue focusing on the use of the *synchronized* keyword